

Accélération d'une FFT sur un processeur RISC-V - FFTurbo

Lucas BELLIER¹ et Théophile DECONCHE²

¹IMT Atlantique Brest : lucas.bellier@imt-atlantique.net

²IMT Atlantique Nantes : theophile.deconche@imt-atlantique.net

Abstract

Ce rapport a pour objectif de présenter notre démarche et nos propositions pour l'optimisation de l'exécution d'une FFT 512 points sur le processeur RISC-V Ariane qui intègre le cœur CVA6. L'optimisation passe dans un premier temps par des changements sur le processeur puis des changements logiciels afin d'exploiter un maximum les modifications matérielles. Ce rapport rend compte de nos résultats pour la participation au concours annuel co-organisé par Thales, le GRD SOC^{2a} et le CNFM^b sur l'architecture RISC-V. Nos optimisations se basent sur un coprocesseur couplé au processeur permettant l'exécution de nouvelles instructions. Les premiers ajouts visaient à effectuer les papillons de la FFT de façon matérielle. Les suivants implémentent un buffer permettant la sauvegarde des données très utilisées. Finalement la création d'instructions de calcul d'adresses de réarrangement nous ont permis d'atteindre une accélération de x6,74.

^aGroupe de recherche System On Chip - Systèmes embarqués et Objets Connectés

^bGroupe de recherche d'intérêt public pour la coordination nationale de la formation en micro électronique et en nanotechnologies

1. INTRODUCTION

1.1. Concours

Le concours RISC-V Student Contest en est à sa 6^{ème} édition, il rassemble chaque année plusieurs équipes d'étudiants d'universités françaises qui s'affrontent sur un sujet précis. Pour cette 6^{ème} édition le sujet choisi est l'optimisation d'une FFT 512 sur un processeur Ariane qui intègre le cœur CVA6 dans sa version 32 bits. Le code d'origine est disponible ici : [1].

1.2. Environnement

1.2.1. Processeur

Le processeur Ariane est un processeur open source de taille moyenne développé par ETH Zürich, il est capable d'exécuter des systèmes d'exploitation avec plusieurs niveaux de privilège. Repris par la Fondation OpenHW, le processeur a été décliné en une version 64 bits puis compacté en une version 32 bits par Thales. C'est sur cette version 32 bits que le concours se base.

L'implémentation du processeur se fait sur un FPGA plus précisément sur une carte Digilent Zybo Z7-20. La carte d'évaluation permet la programmation du FPGA via une interface JTAG et la connexion d'une console pour la visualisation des résultats via une interface UART. La synthèse ainsi que le placement et routage du circuit sont faits avec Vivado [2].

1.2.2. Application

La FFT de 512 points est exécutée par le processeur comme une application bare-metal. Celle-ci est compilée par une toolchain GCC spécifique au set d'instruction RISC-V. Les différentes instructions propriétaires ont directement été décrites dans les fichiers source de l'application pour éviter toute modification de la toolchain et garder l'environnement docker de compilation fourni par les organisateurs.

1.2.3. Simulation

Pour permettre des modifications, des tests et un débogage plus rapide du processeur, un environnement de simulation dédié est fourni par les organisateurs. Il permet la simulation de l'exécution de l'application sur le processeur ainsi que la visualisation des résultats sortants de l'interface UART. Nous avons modifié en partie cet environnement pour ajouter nos instructions propriétaires aux logs de simulation afin de faciliter leur lecture.

2. ANALYSE DE LA FFT 512

La FFT 512 points effectuée par l'application se base sur le code source kissfft. Cette implémentation utilise des paramètres spécifiques qui donnent certaines propriétés à l'exécution de la FFT :

- `N = 512` — La FFT est de 512 points.
- `KISSFFT_DATATYPE=int16_t` — Les nombres complexes sont enregistrés comme 2 entiers signés de 16 bits.
- `g_factors[10] = {4, 128, 4, 32, 4, 8, 4, 2, 2, 1}` — Les étages de la FFT utilisent des papillons de taille 2 ou 4.

Ces paramètres nous permettent de comprendre l'ordre d'exécution de la FFT et l'agencement des données en mémoire.

2.1. Agencement des données

Le type des données étant `KISSFFT_DATATYPE=int16_t`, un nombre complexe quelconque est alors défini comme `struct {int16_t r; int16_t i;}`. Cette structure organise chaque nombre complexe comme un vecteur de 32 bits.

2.2. Taille des papillons

L'organisation des étages met en évidence que seulement 2 types de papillons sont utilisés. Ceux de taille 2 et ceux de taille 4. Chaque papillon utilise une quantité de nombres complexes, 3 pour un de taille 2 et 7 pour un de taille 4.

2.3. Ordre d'exécution

KissFFT utilise une méthode récursive pour le calcul de la FFT, l'exécution des papillons dans les étages suit alors la forme d'un arbre. La Figure 1 montre l'exemple d'une FFT récursive avec seulement des papillons de taille 2. En noir est écrit le type d'étage et en rouge l'ordre d'exécution des papillons.

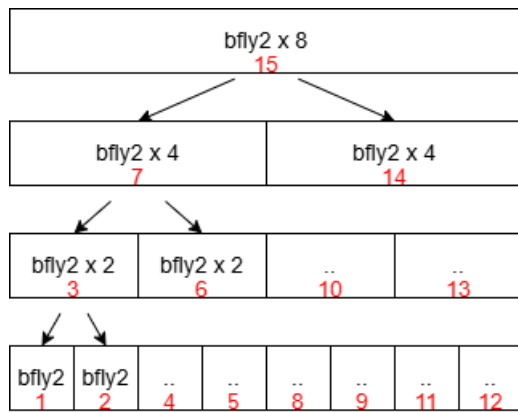


Figure 1. Exemple d'exécution récursive

3. COPROCESSEUR ET PAPILLONS

Notre première modification sur le processeur a été d'intégrer les papillons de taille 2 et 4 dans un coprocesseur utilisant l'interface CVXIF du CVA6. Pour utiliser ce coprocesseur nous avons créé 9 instructions propriétaires qui permettent de configurer et d'utiliser le coprocesseur. Comme noté dans la partie agencement de données, les nombres complexes peuvent être considérés comme des vecteurs de 32 bits. De plus, comme ces vecteurs sont alignés en mémoire, ils peuvent donc être chargés en une seule instruction LW (Load Word) au lieu de 2 instructions LH (Load Half) consécutives et de même pour le stockage. Les instructions suivantes utilisent aussi ce principe, chaque élément des papillons est alors un vecteur de 32 bits contenant la partie imaginaire sur les 16 MSB et la partie réelle sur les 16 LSB.

- BFLY_CFG
- BFLY_SET_F0F2
- BFLY_SET_F1F3
- BFLY_SET_W1W3
- BFLY_SET_W2
- BFLY_GET_F0
- BFLY_GET_F1
- BFLY_GET_F2
- BFLY_GET_F3

3.1. BFLY_CFG(rs1)

Cette instruction configure le coprocesseur en sélectionnant le type de papillon à exécuter (taille 4 ou taille 2) et l'inversion des opérations pour une FFT inverse. Si le bit 3 de rs1 est à 1, le mode sélectionné est un papillon de taille 4, s'il est à 0 c'est un papillon de taille 2. Le bit 2 contrôle le signal d'inversion (inv), permettant une inversion nécessaire dans les opérations internes d'un papillon de taille 4. Au long de cet article si ce n'est pas précisé le code provient du fichier bfly sv dans cvxif_example.

```
inv <= X(rs1)[2]
bfly_mode <= X(rs1)[3]
```

3.2. BFLY_SET_F0F2(rs1, rs2)

Cette instruction charge deux opérandes et effectue leur mise à l'échelle. Pour un papillon de taille 2, elle charge le premier et le second élément. Pour une taille 4, elle charge le premier et le troisième élément. Les résultats sont stockés dans les registres internes div0_ff et div2_ff.

```
div0_ff <= fixdiv(X(rs1)[31:0], bfly_mode)
div2_ff <= fixdiv(X(rs2)[31:0], bfly_mode)
```

3.3. BFLY_SET_F1F3(rs1, rs2)

Exclusivement pour les papillons de taille 4, cette instruction charge et met à l'échelle les deuxième et quatrième éléments. Les résultats sont stockés dans les registres div1_ff et div3_ff.

```
div1_ff <= fixdiv(X(rs1)[31:0], bfly_mode)
div3_ff <= fixdiv(X(rs2)[31:0], bfly_mode)
```

3.4. BFLY_SET_W1W3(rs1, rs2)

Exclusivement pour les papillons de taille 4, charge les premier et troisième twiddle_factors.

```
tw1 <= X(rs1)[31:0]
tw3 <= X(rs2)[31:0]
```

3.5. BFLY_SET_W2(rs1)

Charge l'unique twiddle_factor pour une taille 2, ou le second pour une taille 4. Elle sert également de déclencheur pour propager les données dans le pipeline vers l'étage de multiplication.

```
tw2 <= X(rs1)[31:0]
mul0_ff <= div0_ff
mul1_ff <= mul(div1_ff, tw1)
mul2_ff <= mul(div2_ff, tw2)
mul3_ff <= mul(div3_ff, tw3)
```

3.6. BFLY_GET_Fi(rd)

Récupère les résultats calculés à la sortie du dernier étage. Nécessite 2 appels (F0, F1) pour la taille 2, et 4 appels pour la taille 4.

```
X(rd) <= bfly_out_i
```

3.7. Modifications logicielles

Après l'implémentation de ces instructions, seules les fonctions **kf_bfly2** et **kf_bfly4** de **kiss_fft.c** ont été modifiées afin de tirer parti du coprocesseur. Voici la fonction **kf_bfly2** modifiée à titre d'exemple : [Code 1]

```

static void kf_bfly2_copro(
    kiss_fft_cpx * Fout,
    const size_t fstride,
    const kiss_fft_cfg st,
    int m)
{
    kiss_fft_cpx * Fout2 = Fout + m;
    kiss_fft_cpx * tw1 = st->twiddles;

    uint32_t f0, f2, w;

    f0 = *(uint32_t*)Fout;
    f2 = *(uint32_t*)Fout2;
    w = *(uint32_t*)tw1;

    BFLY_SET_F0F2(f0, f2);
    BFLY_SET_W2(w);

    while (--m) {
        tw1 += fstride;

        uint32_t f0_next = *(uint32_t*)(Fout + 1);
        uint32_t f2_next = *(uint32_t*)(Fout2 + 1);
        uint32_t w_next = *(uint32_t*)tw1;

        BFLY_GET_F0(*(uint32_t*)Fout);
        BFLY_GET_F1(*(uint32_t*)Fout2);

        f0 = f0_next;
        f2 = f2_next;
        w = w_next;

        BFLY_SET_F0F2(f0, f2);
        BFLY_SET_W2(w);

        ++Fout;
        ++Fout2;
    }
    BFLY_GET_F0(*(uint32_t*)Fout);
    BFLY_GET_F1(*(uint32_t*)Fout2);
}

```

Code 1. Fonction kf_bfly2_copro

3.8. Résultats

L'utilisation de ces instructions propriétaires permet d'éviter un grand nombre d'instructions qui composait les papillons. Plus particulièrement ces instructions permettent de simplifier les additions, soustractions, multiplications et arrondi. Voici l'accélération obtenue en simulation [Tableau 1] :

Métrique	Baseline	Coprocasseur	Variation (%)
Instructions	124 142	51 839	-58,24
Cycles	153 490	74 417	-51,52
CPI	1,24	1,44	-

Table 1. Résultats de la version Coprocasseur

4. BUFFER TWIDDLE FACTORS

Lorsqu'on regarde l'ordre d'exécution des papillons on peut remarquer que certains papillons réutilisent des twiddle factors. Dans la FFT de 512 points, les facteurs sont parfois sollicités de manière très répétée. Par exemple, tous les papillons de taille 2 partagent le même facteur. Pour éviter la latence des lectures en mémoire, nous avons intégré trois buffers de 32 mots de 32 bits (tw1_buffer, tw2_buffer, tw3_buffer) dans l'architecture du coprocasseur. La taille des buffers se base sur l'avant dernier étage de la FFT ou se répète 4 fois 32 papillons de taille 4 sachant que chaque papillon utilise 3 twiddle factors. Pour utiliser ces buffers, nous avons fait évoluer les instructions de configuration et d'écriture.

• BFLY_CFG • BFLY_SET_W1W3 • BFLY_SET_W2

4.1. Évolution de BFLY_CFG(rs1)

Configure également le fonctionnement des buffers internes. Le bit 0 (fill_buffer) autorise l'écriture, le bit 1 réinitialise le pointeur tw_index, et les bits 11 à 4 définissent la limite m du buffer.

```

fill_buffer <= X(rs1)[0]
if (X(rs1)[1]) tw_index <= 0
inv <= X(rs1)[2]
bfly_mode <= X(rs1)[3]
m <= X(rs1)[11:4]

```

4.2. Évolution de BFLY_SET_W1W3(rs1, rs2)

Si fill_buffer est actif, enregistre rs1 et rs2 dans les buffers respectifs tw1_buffer et tw3_buffer à l'index courant.

```

if (fill_buffer) tw1_buffer[tw_index] <= X(rs1)[31:0]
if (fill_buffer) tw3_buffer[tw_index] <= X(rs2)[31:0]

```

4.3. Évolution de BFLY_SET_W2(rs1)

Si fill_buffer est actif, enregistre rs1 dans tw2_buffer. Gère ensuite l'incréméntation ou la remise à zéro du pointeur tw_index en fonction de la limite m .

```

if (fill_buffer) tw2_buffer[tw_index] <= X(rs1)[31:0]
if (tw_index == m-1)
    tw_index <= 0
else
    tw_index <= tw_index + 1

```

4.4. Modifications logicielles

Pour pouvoir utiliser ces buffers sur tous les types de papillons (pas seulement de taille 2), il faut rapprocher la réutilisation des **twiddle factors**. Initialement, **kiss_fft.c** fonctionne de façon récursive, ce qui espace les réutilisations (il faut repartir de 0 dans une autre branche et la remonter avant de réutiliser un facteur enregistré). Nous avons donc modifié le code pour que l'exécution soit itérative. Ainsi, les répétitions se font sur un même étage de papillons, tous les papillons de même taille sont à suivre, ce qui remplit plus vite les buffers sur les étages les plus hauts.

La Figure 2 montre le nouvel ordre d'exécution. Pour un même type de papillon (ex bfly2 x 4) les twiddle factors sont les mêmes. Les même types étant maintenant les uns à la suite des autres il est possible de garder les twiddle factors dans un buffer.

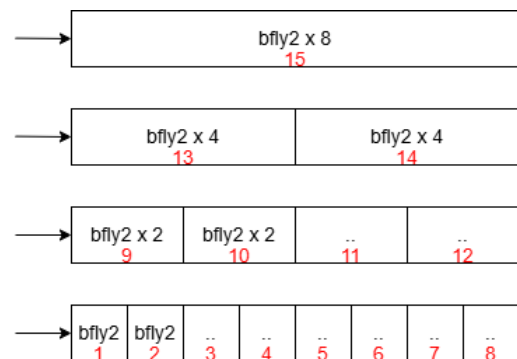


Figure 2. Exemple d'exécution itérative

5. CALCUL D'ADRESSES

Le passage d'une FFT récursive à itérative implique une nouvelle gestion du réarrangement des données de la FFT. Initialement le réarrangement était intégré dans la récursivité mais doit être pris en compte dans cette version itérative. Ce réarrangement correspond au bit-reversal ou au digit-reversal et peut être effectué de façon logicielle dans ce cas cela nécessite un grand nombres de modulo. Cependant ce type d'opération peut être effectué de façon combinatoire avec de nouvelles instructions.

5.1. BFLY_REV_RST(rs1)

Prépare le coprocesseur au réarrangement des données. Sauvegarde l'adresse de base du tableau destination dans dst_ptr et réinitialise le compteur d'itérations r_counter.

```
dst_ptr <= X(rs1)[31:0]
r_counter <= 0
```

5.2. BFLY_REV(rd)

Permute les bits du compteur interne pour générer l'index sur 9 bits (spécifique FFT 512) et calcule l'adresse finale à partir de dst_ptr. Le résultat est stocké dans rd et le compteur est incrémenté.

```
let next_index = {21'b0, r_counter[1:0], r_counter[3:2],
                  r_counter[5:4], r_counter[7:6], r_counter[8], 2'b0}
X(rd) <= next_index + dst_ptr
r_counter <= r_counter + 1
```

5.3. Résultats

Voici l'accélération obtenue en simulation [Tableau 2]. On remarque que puisque cette amélioration diminue le nombre de chargements, on diminue le nombre de cycles à attendre les données et donc le CPI.

Métrique	Baseline	Buffer	Variation (%)
Instructions	124 142	17 969	-85,53
Cycles	153 490	22 781	-85,16
CPI	1,24	1,27	-

Table 2. Résultats de la version Buffer Twiddle Factors

6. RÉSULTATS FINAUX

Nous avons diminué la fréquence de fonctionnement de 40 MHz à 32 MHz, soit la diminution maximale de 32 MHz : [Tableau 3], d'après cva6_fpga.timing.rpt.

	Baseline	FFTurbo	Variation (%)
Slack (MET) (ns)	0,409	0,100	-75,55
Fréquence Maximale (MHz)	40	32	-20,00

Table 3. Fréquence Maximale

L'ajout de notre coprocesseur a beaucoup augmenté le nombre de Flip-Flops et surtout le nombre d'unité DSP : [Tableau 4], d'après cva6_fpga.utilization.rpt.

Métrique	Baseline	FFTurbo	Différence	Variation (%)
Total LUTs	19 804	26 538	+6 734	+34,00
Flip-Flops	14 624	21 394	+6 770	+46,29
DSP	4	23	+19	+475,0
RAMB36	60	66	+6	+10,00

Table 4. Utilisation des ressources

Récapitulatif des résultats en fonction des versions, en simulation [Tableau 5] :

Métrique	Baseline	Coprocesseur	Buffer
Instructions	124 142	51 839	17 969
Variation (%)	0	-58,24	-85,53
Cycles	153 490	74 417	22 781
Variation (%)	0	-51,52	-85,16
Speedup	1	2,06	6,74

Table 5. Résultats pour chaque version

La Figure 3 montre le nombre d'instructions utilisées par le code fourni et notre solution côte à côte. On constate une nette différence sur les instructions de chargement et de stockage : les LH et SH ainsi que les instructions de décalage servant à gérer ces chargements et stockages. C'est dû à la fois à l'utilisation des LW et SW pour charger des données par 32 bits mais aussi grâce aux buffers qui suppriment les chargements des twiddle factors. On remarque également que les instructions mathématiques (ADD, MUL) ont été fortement réduites car les calculs sont faits par le coprocesseur (auquel correspondent les nouvelles instructions custom surnommées OFFLOAD).

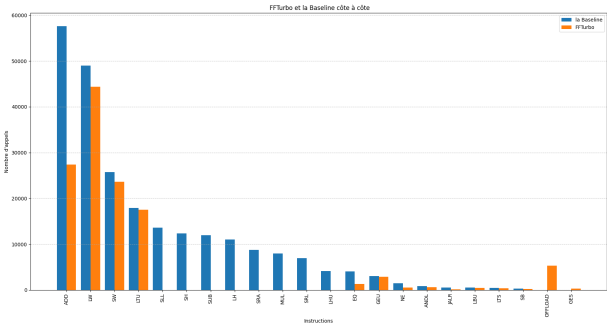


Figure 3. Comparaison du nombre d'instructions entre la Baseline et FFTurbo

7. CONCLUSION

Ces améliorations ont permis d'obtenir un speedup de **x6,74** sur l'application kiss_fft. Les idées majeures étant le chargement des données par mots entiers de 32 bits pour diviser par 2 le nombre de chargements, utiliser la redondance des données (les sorties deviennent les entrées et la réutilisation des facteurs) pour stocker temporairement dans un buffer les résultats intermédiaires et éviter de stocker puis recharger les données depuis la mémoire.

7.1. Remerciements

Nous tenons à remercier l'équipe d'enseignants de l'école IMT Atlantique et du laboratoire Lab-STICC qui nous a accompagnés durant ce concours : Ali Al Ghouwayel, Amer Baghdadi et Stefan Weithoffer.

Nous souhaitons également adresser nos remerciements aux organisateurs de la compétition pour nous avoir donné l'opportunité de participer à ce projet.

[3]

■ REFERENCES

[1] Thales, *National risc-v student contest cv32a6*. 2026. [Online]. Available: <https://github.com/ThalesGroup/cva6-softcore-contest/>.

[2] AMD-Xilinx, *Vivado design suite 2024.1*. [Online]. Available: <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vivado-design-tools/2024-1.html>.

[3] G. Jimenez and E. Gracidas, *Rho-class: A LaTeX2e template*, version 3.0.0, Licence: Creative Commons CC BY 4.0, 2026. [Online]. Available: <https://github.com/MemoJimenez/Rho-class>.